# Lantern: Learned Prefetching for Tiered Key-Value Stores

**Noor Haddad**
University of Toronto
noor@cs.example.edu

**Mateus Lacerda**
University of Toronto
mlacerda@cs.example.edu

**Yuki Tanabe**
Kioxia Research
y.tanabe@example.com

**Rachel Abernethy**
University of Toronto
rabernethy@cs.example.edu

ABSTRACT Key-value stores increasingly place hot data in DRAM and the long tail on NVMe flash. Replacement policies decide what to keep in the fast tier, but they act only after a miss, and the miss is what costs a flash read. We present Lantern, a prefetcher that predicts, for each key that is accessed, the small set of keys likely to follow it within the next millisecond and promotes them ahead of demand. A gradient-boosted model over co-access features fits in 6 MB per shard, is retrained online every few minutes, and gates every prefetch by a cost-aware threshold so that wrong guesses never evict useful data. On four production-derived traces Lantern lowers P99 read latency by 31–47% over ARC and by 18% over the best sequence-model prefetcher, at under 3% CPU overhead.

CCS CONCEPTS Information systems → Key-value stores; Hierarchical storage management; Computing methodologies → Supervised learning.
KEYWORDS prefetching, tiered storage, key-value stores, learned systems

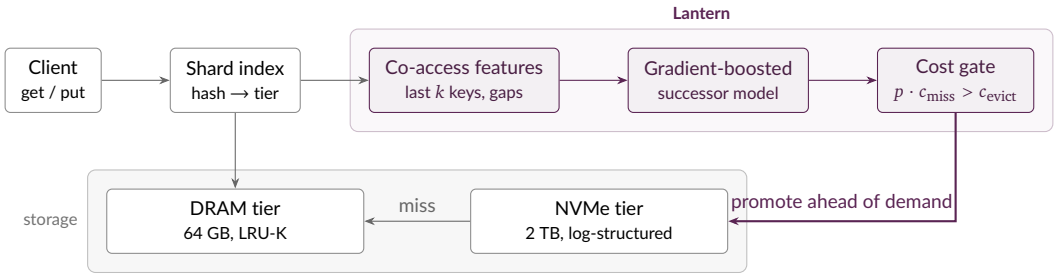



**Figure 1.** Lantern sits beside the shard index. Every access updates co-access features; the model proposes likely successors and a cost gate promotes only those whose expected saving exceeds the cost of the eviction they cause.

## 1 Introduction

Large key-value deployments no longer fit in memory. Operators place a few percent of the data in DRAM and the rest on NVMe flash, and rely on a replacement policy to keep the hot set in the fast tier [9, 10]. Replacement has been studied for decades, from LRU-K to ARC to learned approximations of Belady's algorithm [5, 6, 8], and modern policies capture most of what recency and frequency can explain.

What no replacement policy can do is avoid the first miss. A key that has not been accessed recently will be read from flash, at roughly a hundred times the latency of DRAM, and on production traces these cold reads dominate the tail [10]. Yet many of them are predictable: keys are accessed in groups that follow application logic, such as a user record followed by its settings and its recent sessions. A prefetcher that learned these groups could promote the followers while the leader is being served.

Prefetching for storage is not new, and sequence models trained on access logs predict the next key well in the laboratory [3, 7]. They are hard to deploy for three reasons. They are large and slow to run per request; they drift as applications change; and a wrong prefetch is not free, because it evicts a key that might have been used. We address each in turn:

- A compact *successor model* built on co-access features and gradient-boosted trees [4], small enough to run inline (Section 3).
- *Online retraining* on a sliding window, which tracks workload drift without an offline pipeline.
- A *cost gate* that admits a prefetch only when its expected saving exceeds the expected cost of the eviction it causes (Section 3.3).

Section 5 evaluates Lantern on four traces and shows where it does not help.

## 2 Background and Motivation

### 2.1 Tiered key-value stores

A tiered store keeps an index from keys to locations and serves reads from DRAM when it can. On a miss it reads the value from flash, returns it, and admits it to DRAM, evicting another key. Admission and eviction are the policy's two levers; recent work tunes both, for example by filtering one-hit wonders with ghost lists [1].

### 2.2 Where the misses come from

We analysed a week of requests to 32 shards of a social-graph store. After ARC has warmed up, 71% of the remaining DRAM misses are *compulsory within the window*: the key was not accessed in the preceding hour. Of those,

58% are accessed within one millisecond of another key with which they were co-accessed at least five times in the previous day. This is the opportunity Lantern targets: misses that recency cannot predict but co-access history can.

*Why not a sequence model?.* A two-layer LSTM over the last 64 keys predicts the next key with 44% top-10 accuracy on our traces, but costs 38 µs per inference on a server core, more than the DRAM read it is meant to protect. Batched inference amortises the cost but delays the prediction past the moment it is useful.

### 2.3 Workload characterisation

To understand which applications produce learnable successors, we grouped the misses of the *Graph* trace by the key prefix that identifies the calling service. Three services account for 80% of the predictable misses: profile rendering, which reads a user record and then its settings and avatar metadata; feed assembly, which reads a list of post identifiers and then the posts; and notification delivery, which reads a device token after each user record. In all three the successors are fixed by code rather than by user behaviour, which is why they recur so reliably.

The remaining services issue reads whose successors depend on the user, for example the next item in a search result. Their co-access counts are low and their gaps long, so the model assigns them low probabilities and the gate rarely admits them. This division explains a pattern visible throughout our results: Lantern is most effective on traces dominated by a few services with fixed read sequences, and least effective on traces whose accesses are driven by independent user choices.

Two properties of the traces matter for the design. First, the gap between a leader and its successors is short: the median is 140 µs and the 90th percentile is 610 µs, which is why the window $\Delta$ is one millisecond and why a prediction is useless if it arrives after a batching delay. Second, successor sets are small: 90% of leaders have at most twelve successors with a co-access count above five, so a sketch of 32 entries loses little.

### 2.4 Design goals

These observations set four goals. First, a prediction must be ready before the leader's read completes, which on DRAM is a few microseconds; anything slower is a prediction about the past. Second, the model must adapt to changes in application logic without an offline pipeline, because the operators of a storage tier rarely control the applications above it. Third, a wrong prediction must cost less than the benefit of a right one, on every workload, including those with no structure at all. Fourth, the memory spent on metadata must be small compared with the DRAM it protects, or the same bytes would be better spent on a larger cache.

The first goal rules out sequence models on the request path, and the second rules out models trained once and shipped. The third is the one most prior prefetchers do not address: they are evaluated by accuracy, while the quantity that matters is the change in tail latency once mistakes are paid for. Lantern is designed backwards from that quantity.

## 3 design

Lantern runs inside each shard and observes every access. It has three parts: a feature extractor, a successor model and a cost gate (Figure 1).

### 3.1 Co-access features

For each key $k$ the extractor keeps a bounded *successor sketch* $S(k)$: the up to 32 keys that most often followed $k$ within a window $\Delta$, with counts and the median gap between the two accesses. Sketches are updated in constant time with a space-saving counter and are stored next to the index entry, so a lookup costs no extra random access. For a candidate pair $(k, j)$ with $j \in S(k)$ the features are the co-access count and its decay-weighted variant, the median gap, the recency and frequency of $j$, whether $j$ is already in DRAM, and the size of its value.

### 3.2 Successor model

A single gradient-boosted model per shard scores each candidate with the probability $p(j \mid k)$ that $j$ is read within $\Delta$. We use 64 trees of depth six, which take 1.9 µs to score 32 candidates and occupy 6 MB. The model is retrained every five minutes on labels collected from the previous window, which are free: we observe whether each candidate was in fact read. Retraining runs on a background core and swaps the model atomically.

### 3.3 Cost gate

A prefetch is useful only if its expected benefit outweighs the harm it does. Promoting $j$ saves a flash read with probability $p(j \mid k)$, but it evicts the DRAM victim $v$, which may itself be read soon. Let $c_{\text{miss}}$ be the cost of a flash read and $q(v)$ the probability that $v$ is read before it would have been evicted anyway, estimated from its ARC list position. The gate admits $j$ when

$$p(j \mid k)\, c_{\text{miss}} \;>\; q(v)\, c_{\text{miss}} + c_{\text{io}}, \tag{1}$$

where $c_{\text{io}}$ charges the flash bandwidth the prefetch consumes. Because both sides scale with $c_{\text{miss}}$, the rule reduces to $p(j \mid k) - q(v) > c_{\text{io}}/c_{\text{miss}}$, a threshold that we set once per device. Algorithm 1 gives the complete access path.

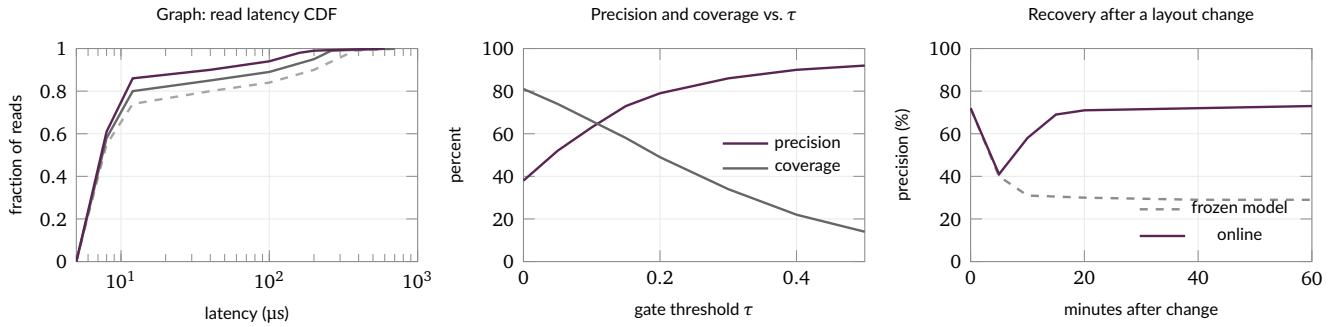



**Figure 2.** Left: CDF of read latency on *Graph* for ARC (dashed), ARC with an LSTM prefetcher (grey) and Lantern. Centre: the gate threshold trades coverage of misses for precision of prefetches; the device-derived value is $\tau = 0.15$. Right: precision after the *Shop* catalogue changes layout at minute zero, with a frozen model and with five-minute online retraining.

**Algorithm 1:** Access path with Lantern prefetching.

**Input:** accessed key $k$, shard index $I$, model $M$, threshold $\tau$

serve $k$ from DRAM or flash and update $I$;
update successor sketches of the last keys with $k$;
$C \leftarrow S(k) \setminus \text{DRAM}$ ; *// candidates not yet hot*
**foreach** $j \in C$ *in batches of 32* **do**
  $p_j \leftarrow M(\phi(k, j))$;
**foreach** $j \in C$ *by decreasing* $p_j$ **do**
  $v \leftarrow$ next eviction victim in DRAM;
  **if** $p_j - q(v) > \tau$ **then**
    issue asynchronous read of $j$; promote on completion;
  **else break** *// later candidates score lower*;
record $(k, C)$ as pending labels for retraining;

## 4 Implementation

We built Lantern as a library of about 3,400 lines of C++ linked into the storage server. Sketches live in a slab next to each hash-index bucket, so updating them touches a cache line that the lookup has already loaded. The model is compiled to a flat array of split thresholds and evaluated with branch-free code, which is what keeps scoring under two microseconds; a generic tree library was six times slower on the same model.

Prefetches are issued through the same asynchronous I/O queue as demand reads but at a lower priority, so a burst of prefetches can never delay a client read. A promoted key enters DRAM at the middle of the ARC recency list rather than at its head: if the prediction was wrong the key leaves quickly, and if it was right the first real access moves it to the head as usual. Labels for retraining are written to a ring buffer of 4 million entries per shard, and a background thread fits the next model with the same feature code that serves requests, which removes a whole class of training–serving skew.

*Failure handling.* If retraining fails or produces a model whose validation precision falls below the current one, the shard keeps the current model and logs the event. If the I/O queue exceeds a depth limit, the gate raises its threshold until the queue drains, so prefetching backs off under load instead of adding to it.

**Table 1.** P99 read latency in microseconds and DRAM hit ratio in percent. Lower latency and higher hit ratio are better; the best value in each column is set in bold.

| | P99 latency (µs) | | | | Hit |
|---|---|---|---|---|---|
| Policy | Graph | Session | Shop | YCSB-B | (%) |
| LRU-K | 412 | 338 | 501 | 286 | 81.4 |
| ARC | 377 | 305 | 466 | 261 | 83.9 |
| ARC + ghost lists | 351 | 297 | 440 | 255 | 85.2 |
| ARC + LSTM | 262 | 248 | 371 | 249 | 88.0 |
| Lantern | **201** | **209** | **318** | **246** | **91.3** |

## 5 Evaluation

*Setup.* We replay traces against a prototype built on a log-structured store with a 64 GB DRAM tier and a 2 TB NVMe tier on a 32-core server. We compare against LRU-K and ARC without prefetching [5, 6], ARC with a ghost-list admission filter [1], and ARC with an LSTM prefetcher batched every 200 µs, the best configuration of [7]. Traces are a social-graph store (*Graph*), a session cache (*Session*), an e-commerce catalogue (*Shop*) and YCSB workload B with Zipfian keys [2].

*Tail latency.* Table 1 reports P99 read latency. Lantern lowers it by 47% over ARC on *Graph*, where co-access groups are strongest, and by 31% on *Shop*. On YCSB-B the gain is within noise: keys are drawn independently, so there are no successors to learn, and the cost gate correctly declines almost every prefetch. The latency CDF in Figure 2 shows where the gain comes from: the median is unchanged, while the fraction of reads served from flash falls from 26% to 14%. This is the behaviour we want from a prefetcher on a workload it cannot help.

*Memory pressure.* Figure 3 varies the DRAM tier from 1% to 16% of the data. The benefit of prefetching shrinks

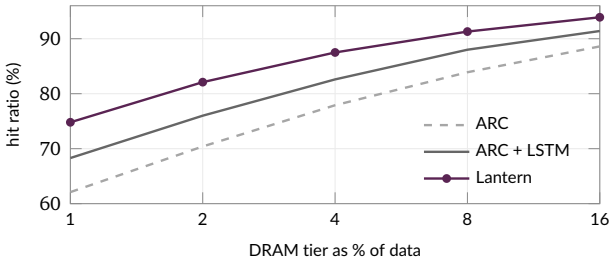



**Figure 3.** DRAM hit ratio on *Graph* as the fast tier grows. Prefetching matters most when DRAM is scarce: at 1% of the data Lantern matches ARC with four times the memory.

**Table 2.** Ablation on *Graph*: P99 latency, prefetch precision and the flash bandwidth spent on prefetches that were never read.

| Variant | P99 (µs) | Prec. (%) | Waste (MB/s) |
|---|---|---|---|
| Full Lantern | **201** | **73** | **18** |
| without cost gate | 236 | 44 | 71 |
| without online retraining | 228 | 61 | 29 |
| sketch size 8 instead of 32 | 219 | 70 | 21 |
| LSTM instead of trees | 247 | 69 | 24 |

as memory grows, because more successors are already resident, but it never turns negative: the gate keeps wrong guesses from displacing useful keys. With DRAM at 1% of the data, Lantern reaches the hit ratio that ARC needs 4% to achieve.

*Accuracy and waste.* Of the prefetches Lantern issues on *Graph*, 73% are read within $\Delta$; the LSTM prefetcher, which has no gate, reaches 41% and wastes 2.6 times as much flash bandwidth. Raising the threshold $\tau$ in (1) trades coverage for precision smoothly; we found the value derived from device costs within 5% of the best setting on every trace (Figure 2, centre).

*Overhead.* Sketch updates and inference add 2.4 µs to the median request and use 2.7% of the server's CPU. Retraining takes 11 s per shard every five minutes on a background core. Sketches add 410 B per hot key, or 3.1% of the DRAM tier.

*Ablation.* Table 2 removes one component at a time. The cost gate matters most: without it precision drops to 44% and the wasted bandwidth quadruples, which in turn slows demand reads that share the device. Online retraining is the second largest contributor. Replacing the trees by the LSTM keeps precision almost unchanged but raises latency, because inference no longer fits on the request path and predictions arrive late.

*Drift.* When we switch the *Shop* trace to a new catalogue layout halfway through, a frozen model loses 60% of its benefit within an hour, while the online model recovers its accuracy within two retraining windows (Figure 2, right).

## 6 Discussion and limitations

*When prefetching cannot help.* Lantern learns successors from history, so it helps only where accesses come in groups that repeat. Uniform or purely Zipfian workloads, like YCSB-B, offer nothing to learn; the gate makes Lantern nearly inert on them, but it still pays the cost of maintaining sketches. A deployment that knows its workload has no structure should disable it.

*Cross-shard successors.* Sketches are local to a shard. When an application reads a key on one shard and its successor lives on another, Lantern cannot see the pair. On our traces 14% of co-access pairs cross shards; capturing them requires exchanging sketches between shards, which we leave to future work.

*Cost model.* The gate in (1) assumes a fixed flash read cost. Under heavy load the true cost rises with queue depth, and a prefetch that was cheap at idle becomes expensive. The back-off described in Section 4 handles the extreme case; a gate that reads the current queue depth would handle the intermediate ones better.

## 7 Related work

Replacement policies decide what to keep after an access [5, 6], and learned policies approximate the optimal offline decision [8]. Admission filters decide what to let in [1]. Lantern is complementary: it acts *before* an access, and it reuses the policy's own estimate of a victim's value in its gate. Learned prefetchers for memory and storage predict the next address or key with sequence models [3, 7]; we trade some accuracy for a model that is cheap enough to run inline and retrain continuously, and we add the cost model that makes wrong predictions safe. Placement in tiered stores [9] decides where a key lives in steady state, while prefetching responds to the next millisecond.

## 8 Conclusion

Cold reads dominate the tail of tiered key-value stores, and many of them are predictable from co-access history. Lantern predicts them with a small model that runs inline, retrains online, and prefetches only when the expected saving exceeds the cost of the eviction. It lowers P99 latency by up to 47% and never hurts a workload it cannot help. We plan to share sketches across shards for keys whose successors live elsewhere.

*Acknowledgements.* We thank the anonymous reviewers and the operators who shared anonymised traces. This work was supported by a fictional research grant.

## REFERENCES


[1] H. Brandão, A. Iyer, and T. Wolfe. “Ghost Lists Revisited: Admission Control for Flash Caches”. In: *Proceedings of the 22nd USENIX Conference on File and Storage Technologies*. 2024, pp. 201–216.

[2] B. F. Cooper et al. “Benchmarking Cloud Serving Systems with YCSB”. In: *Proceedings of the 1st ACM Symposium on Cloud Computing*. 2010, pp. 143–154.

[3] M. Hashemi et al. “Learning Memory Access Patterns”. In: *Proceedings of the 35th International Conference on Machine Learning*. 2018, pp. 1919–1928.

[4] G. Ke et al. “LightGBM: A Highly Efficient Gradient Boosting Decision Tree”. In: *Advances in Neural Information Processing Systems*. Vol. 30. 2017.

[5] N. Megiddo and D. S. Modha. “ARC: A Self-Tuning, Low Overhead Replacement Cache”. In: *Proceedings of the 2nd USENIX Conference on File and Storage Technologies*. 2003, pp. 115–130.

[6] E. J. O’Neil, P. E. O’Neil, and G. Weikum. “The LRU-K Page Replacement Algorithm for Database Disk Buffering”. In: *Proceedings of the ACM SIGMOD International Conference on Management of Data*. 1993, pp. 297–306.

[7] K. Osei, M. Lindqvist, and R. Chen. “Sequence Models for Storage Prefetching: Promise and Pitfalls”. In: *Proceedings of the 15th ACM Workshop on Hot Topics in Storage and File Systems*. 2023, pp. 44–51.

[8] Z. Song et al. “Learning Relaxed Belady for Content Distribution Network Caching”. In: *Proceedings of the 17th USENIX Symposium on Networked Systems Design and Implementation*. 2020, pp. 529–544.

[9] L. Varga, S. Nakamura, and J. Pereira. “Tiering Is Not Caching: Placement Policies for DRAM–Flash Key-Value Stores”. In: *Proceedings of the 2022 USENIX Annual Technical Conference*. 2022, pp. 611–626.

[10] J. Yang, Y. Yue, and K. V. Rashmi. “A Large-Scale Analysis of Hundreds of In-Memory Key-Value Cache Clusters at Twitter”. In: *ACM Transactions on Storage* 17.3 (2021), pp. 1–35.